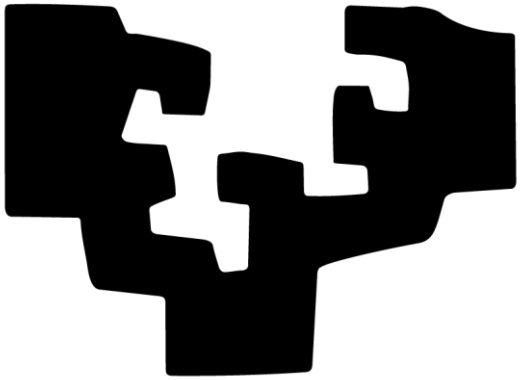
Report

A network file system



EHU

Euskal Herriko Unibertsitatea
University of the Basque Country

Authors: Izaro Uribelarrea, Edoardo Besteghi & Nicolas Jens Beyer

Date: 21-09-2026

The following report evaluates the properties of the SAR/SZA prototype as a distributed system. In the beginning we describe the architecture and functionality of the prototype. Then we define the properties of a Distributed System based on the lecture material. In the following we show which properties are currently fulfilled by our prototypical system and highlight the limitations.

Architecture and Functionality

The SAR prototype represents a Client-Server-Application in Python for file transfer via the Transmission Control Protocol (TCP). Users can list, download, upload and delete files on the server.

Structure

- **szasar.py**: shared helpers. It defines the command constants, *recvline* (reads one CRLF-terminated line) and *recvall* (reads exactly N bytes).
- **cli_fich.py**: an interactive console client with a login loop and a numbered menu.
- **serv_fich.py**: a server that forks one process per client.
- **serv_fich_multithread.py**: a server that starts one thread per client, with a separate directory per user.

Protocol design

- **USER**: The client sends its username to start identification, and the server accepts or rejects it before authentication.
- **PASS**: The client sends the password for the identified user, which authenticates the session and moves it to the main state.
- **LIST**: The client asks for the files in its storage folder, and the server answers with each file's name and size.
- **DOWN**: The client names a file to download, and the server checks that it exists and replies with its size.
- **DOW2**: The client confirms it wants the transfer, and the server then sends the file contents as raw bytes.
- **UPLO**: The client announces the name and size of a file to upload, and the server checks permissions, the size limit and free disk space.
- **UPL2**: After the server has accepted UPLO, the client sends the file contents as raw bytes, which the server stores.
- **DELE**: The client asks the server to delete a named file, which is refused for the read-only anonymous user or if the file does not exist.
- **EXIT**: The client ends the session, and the server closes the connection.

- **Responses**:
  - OK
  - ER<code>

Session management

- The server is stateful and implements a state machine: Identification → Authentication → Main → Downloading/Uploading.

Access control

- Three hard-coded users exist: anonimous, sar and sza. The anonymous user accepts any password and is read-only; upload and delete return an error.

Resource management

- Uploads are limited to 10 MiB.
- Uploads are refused unless 50 MiB of free space would remain after the upload.

Properties

Transparency	Files should have a unique name that doesn't depend on where they physically live, and that name shouldn't change even if the file is moved between servers. The user never needs to know or care which machine holds the data.
Scalability	The system should grow without breaking down. This has two parts: (1) the naming scheme must be structured enough to handle huge numbers of files without running out of names, and (2) caching and replication should spread out the load so a growing number of clients doesn't create a bottleneck at one server.
Robustness	A client crashing shouldn't affect the server, and vice versa. Where possible, operations should be idempotent (repeating the same request has the same effect as doing it once) — this matters because if a client resends a request after a timeout, the server shouldn't end up doing the operation twice with a different result.
Availability & Fault Tolerance	The service should keep working even when parts of it fail, typically achieved through replication. A stronger version of this is allowing a client to keep working on a locally cached copy even when disconnected from the server entirely (as Coda does), and reconciling changes once reconnected.

Consistency	When caching or replication is used, the system should try to behave as close as possible to a single, centralized file system — ideally so that a read always returns the latest write, just like classic Unix semantics. In practice this is hard to fully achieve without sacrificing performance.
Security	Because there's no built-in notion of a "logged-in user" across the network the way there is on a single machine, distributed file systems need an explicit mechanism to prove who's making a request — typically credentials or capabilities — rather than relying on local access-control lists the way a centralized Unix system does.

Evaluation of SAR

PROPERTIES	SZA/SAR	CLOUD SERVICE
Transparency	Partial. Users access files by name but must select a server and transfer files manually.	Hides physical storage locations and provides access through a common interface.
Scalability	Multiple clients can connect, but capacity is limited to one server.	Distributes storage and workload across multiple servers as demand grows.
Robustness	Separate processes isolate sessions, but interrupted uploads can damage files and automatic recovery is absent.	Uses retries, integrity checks and recovery mechanisms to handle interrupted operations.
Availability & Fault Tolerance	Files persist on disk, but server failure prevents remote access. No replication	Uses replication and failover to maintain access during some server failures
Consistency	No locking or conflict detention exists, so clients can overwrite each other's changes	Uses synchronisation, version checks and conflict handling to coordinate updates
Security	Basic authentication is provided, but passwords are hard-coded and non-crypted, and file paths are insufficiently validated	Provides encrypted communication, secure authentication and access permissions
Performance	Limited by a single server and whole-file transfers	Uses caching, load balancing and efficient transfers to reduce delay and bottlenecks
Permanent storage	Files remain on the server's disk between sessions	Stores files persistently, typically with redundant copies.

Namespace and organization	Uses filenames within server directories, with basic organization.	Provides hierarchical folders and metadata to organize and identify files.
Concurrent access	Multiple clients can connect, but conflicting operations are not coordinated.	Coordinates concurrent updates and handles conflicts.
Automatic synchronization	Requires manual uploads and downloads.	Automatically synchronizes changes across connected devices.
Offline operation	Downloaded files can be edited locally but require manual re-upload.	Some services support offline editing and synchronization after reconnection.
Collaboration features	Basic shared access, without integrated co-editing.	Supports sharing and permissions; some services also provide co-editing.
Versioning and recovery	No built-in version history or recovery of deleted files.	Typically supports restoring previous versions and deleted files within retention limits.